Homomorphic Signatures for Polynomial Functions

Dan Boneh and **David Mandell Freeman**

Stanford University, USA

IBM Research
4 November 2010

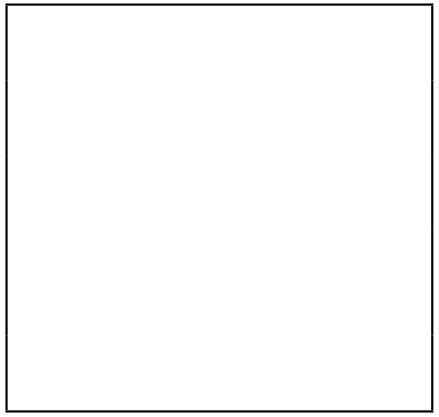
Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

Untrusted DB



Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

$\xrightarrow[\text{grades}]{\text{encrypted}}$

Untrusted DB

Student	CT
1	c_1
2	c_2
$\vdots$	$\vdots$
k	c_k

c_i = encryption of ith score

Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

$\xrightarrow[\text{grades}]{\text{encrypted}}$

Untrusted DB

Student	CT
1	c_1
2	c_2
$\vdots$	$\vdots$
k	c_k

c_i = encryption of ith score

Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

encrypted grades →

Untrusted DB

Student	CT
1	c_1
2	c_2
⋮	⋮
k	c_k

← mean?

c_i = encryption of ith score

Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

$\xrightarrow[\text{grades}]{\text{encrypted}}$

Untrusted DB

Student	CT
1	c_1
2	c_2
$\vdots$	$\vdots$
k	c_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow[c]{}$

c_i = encryption of ith score

c = encryption of mean

Homomorphic Encryption

Homomorphic encryption allows users to delegate computation while ensuring *secrecy*.

encrypted grades →

Untrusted DB

Student	CT
1	c_1
2	c_2
⋮	⋮
k	c_k

← mean?

→ c

c_i = encryption of ith score

c = encryption of mean

- Validity: c decrypts to the correct mean.
- Security: no adversary can obtain any info about scores.
- Length efficiency: c is short.
- Privacy: decrypted mean reveals nothing else about data.

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

Untrusted DB

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\sigma_1 =$ signature on
("grades", 91, "student 1")

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\sigma_1 =$ signature on
("grades", 91, "student 1")

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\sigma_1 =$ signature on
("grades", 91, "student 1")

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow[87.3,\ \sigma]{}$

$\sigma_1 =$ signature on (“grades”, 91, “student 1”)

$\sigma =$ signature on (“grades”, 87.3, “mean”)

Homomorphic Signatures

Homomorphic signatures allow users to delegate computation while ensuring *integrity*.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow[87.3,\ \sigma]{}$

$\sigma_1 =$ signature on (“grades”, 91, “student 1”)

$\sigma =$ signature on (“grades”, 87.3, “mean”)

- Validity: σ authenticates 87.3 as the correct mean.
- Security: no adversary can authenticate a different mean.
- Length efficiency: σ is short.
- Privacy: signature reveals nothing else about data.

More generally: $\mathcal{F}$-homomorphic signatures

- $\mathcal{F}$ is a set of "admissible" functions on messages.
- τ is a "tag" tying together data from the same set.
  (like a filename)
- Given pk, admissible function $f \in \mathcal{F}$, and signatures on data

$$m_1, \ldots, m_k,$$

anyone can compute a valid signature on

$$(\tau, f(m_1, \ldots, m_k), \langle f \rangle),$$

where $\langle f \rangle$ "encodes" the function f.

Applications

What are $\mathcal{F}$-homomorphic signatures good for?

$\mathcal{F}$	Application
Linear functions	Mean Linear least-squares fit (fixed x, variable y) Fourier transforms

Applications

What are $\mathcal{F}$-homomorphic signatures good for?

$\mathcal{F}$	Application
Linear functions	Mean Linear least-squares fit (fixed x, variable y) Fourier transforms
Polynomials (bounded degree)	Standard deviation & higher moments Linear least-squares fit (variable x and y)

Applications

What are $\mathcal{F}$-homomorphic signatures good for?

$\mathcal{F}$	Application
Linear functions	Mean Linear least-squares fit (fixed x, variable y) Fourier transforms
Polynomials (bounded degree)	Standard deviation & higher moments Linear least-squares fit (variable x and y)
Arbitrary circuits	Non-linear estimators and regression Data mining (decision trees, SVM, etc.)

State of the art

What can we do with encrypted or authenticated data?

$\mathcal{F}$	Hom. encryption	Hom. signatures
Linear functions		
Polynomials (bounded degree)		
Arbitrary circuits		

State of the art

What can we do with encrypted or authenticated data?

$\mathcal{F}$	Hom. encryption	Hom. signatures
Linear functions	[GM82], [B88], [P99], others	
Polynomials (bounded degree)	[BGN05], [GHV10] (quadratic)	
Arbitrary circuits	[G09], [DGHV10]	

State of the art

What can we do with encrypted or authenticated data?

$\mathcal{F}$	Hom. encryption	Hom. signatures
Linear functions	[GM82], [B88], [P99], others	[KFM04], [CJL06], [ZKMH07], [BFKW09], [GKKR10], [BF10]
Polynomials (bounded degree)	[BGN05], [GHV10] (quadratic)	
Arbitrary circuits	[G09], [DGHV10]	

State of the art

What can we do with encrypted or authenticated data?

$\mathcal{F}$	Hom. encryption	Hom. signatures
Linear functions	[GM82], [B88], [P99], others	[KFM04], [CJL06], [ZKMH07], [BFKW09], [GKKR10], [BF10]
Polynomials (bounded degree)	[BGN05], [GHV10] (quadratic)	This work
Arbitrary circuits	[G09], [DGHV10]	

State of the art

What can we do with encrypted or authenticated data?

$\mathcal{F}$	Hom. encryption	Hom. signatures
Linear functions	[GM82], [B88], [P99], others	[KFM04], [CJL06], [ZKMH07], [BFKW09], [GKKR10], [BF10]
Polynomials (bounded degree)	[BGN05], [GHV10] (quadratic)	This work
Arbitrary circuits	[G09], [DGHV10]	

Specifically, we construct secure, length-efficient, $\mathcal{F}$-homomorphic signatures for

$$\mathcal{F} = \{\text{polynomials of bounded degree with small coefficients}\}$$

Outline

1. Homomorphic signatures:
   What are they and what are they good for?

2. Linearly homomorphic signatures.

3. Polynomially homomorphic signatures from ideal lattices.

Linearly Homomorphic Signatures

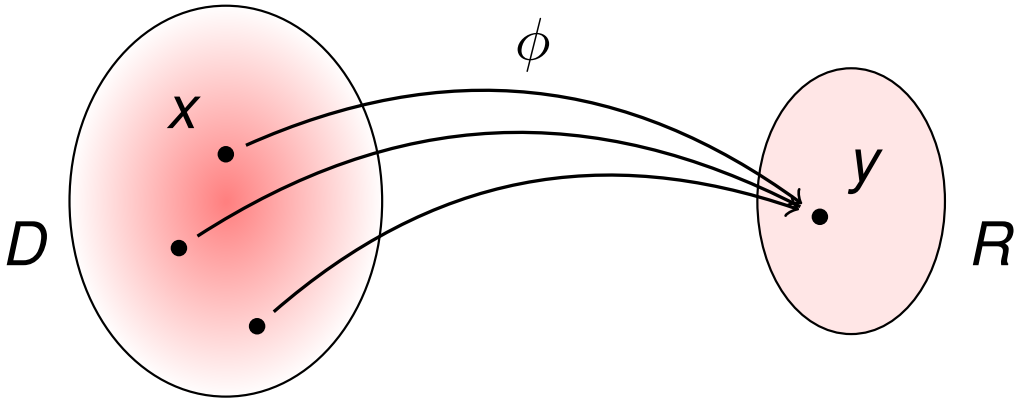
Building block: GPV Signatures

Key idea: preimage sampleable trapdoor function

Building block: GPV Signatures

Key idea: preimage sampleable trapdoor function

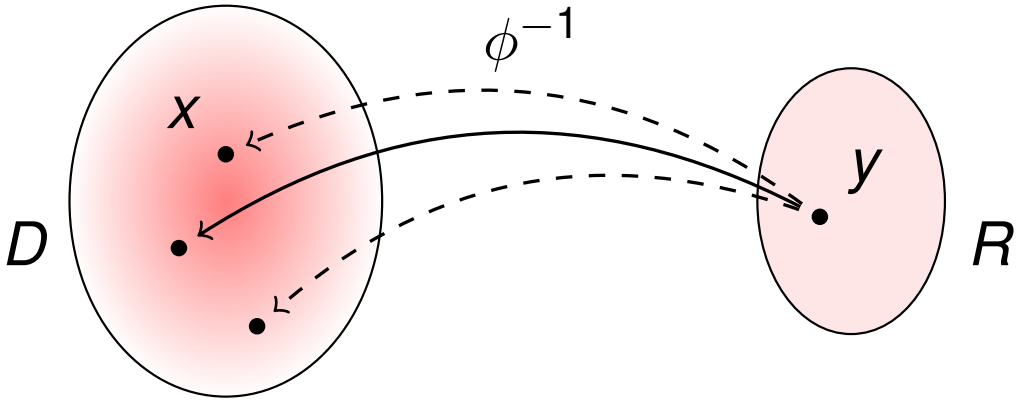
- Public function $\phi\colon D \to R$ with secret "trapdoor" ϕ^{-1}



Building block: GPV Signatures

Key idea: preimage sampleable trapdoor function

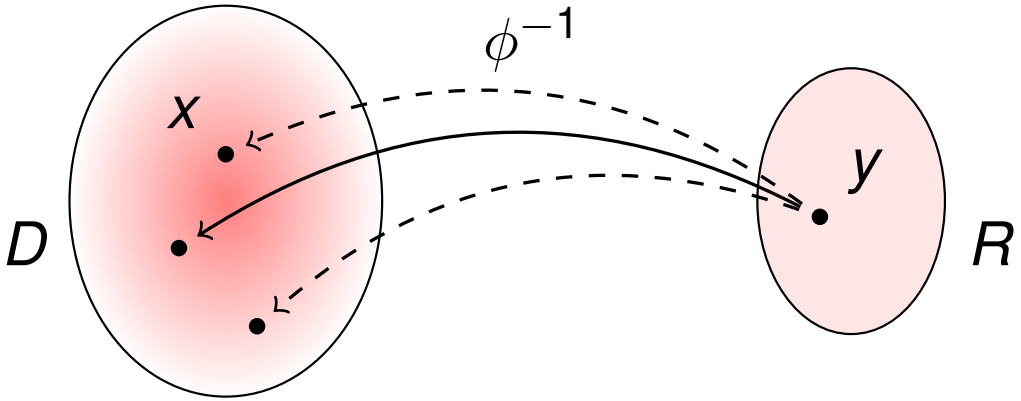
- Public function $\phi\colon D \to R$ with secret "trapdoor" ϕ^{-1}



Building block: GPV Signatures

Key idea: preimage sampleable trapdoor function

- Public function $\phi\colon D \to R$ with secret "trapdoor" ϕ^{-1}



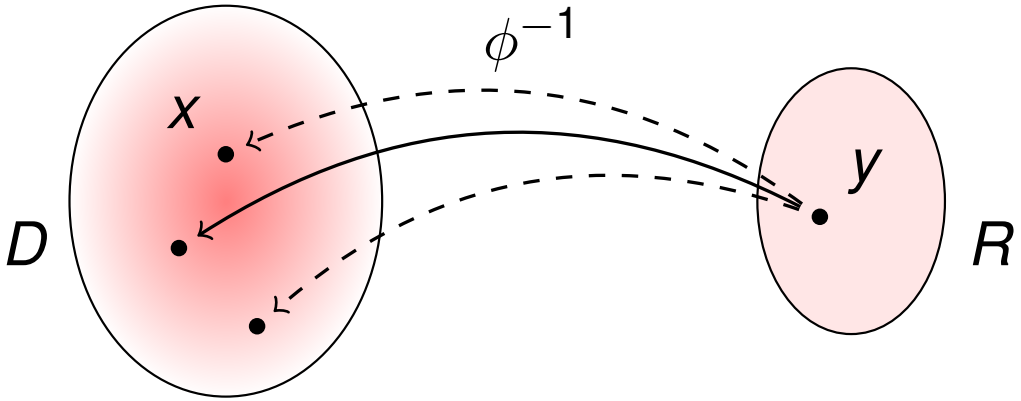
- "Hash and sign:" $pk = \phi$, $sk = \phi^{-1}$, hash $H\colon \{0,1\}^* \to R$

$$\mathrm{Sign}(m) := \phi^{-1}(H(m)).$$

Building block: GPV Signatures

Key idea: preimage sampleable trapdoor function

- Public function $\phi\colon D \to R$ with secret "trapdoor" ϕ^{-1}



- "Hash and sign:" $pk = \phi$, $sk = \phi^{-1}$, hash $H\colon \{0,1\}^* \to R$

$$\mathrm{Sign}(m) := \phi^{-1}(H(m)).$$

- Security (ROM): can generate (x, y) in 2 equivalent ways:

REALITY

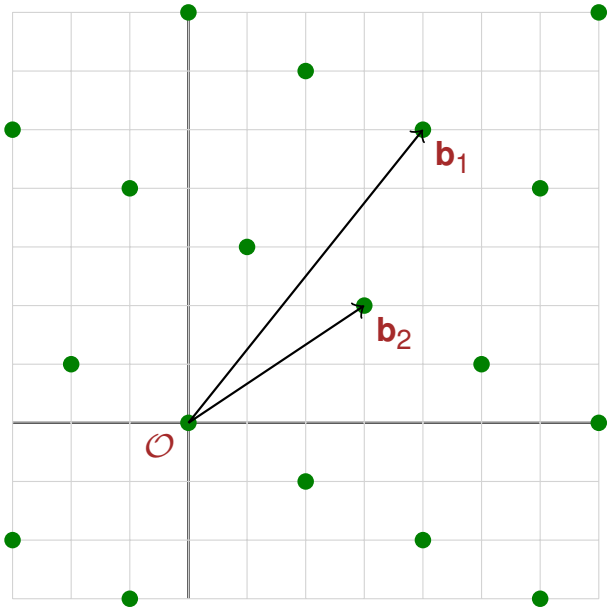
$$x \xleftarrow{\phi^{-1}} y \longleftarrow R$$

PROOF

$$D \longrightarrow x \xrightarrow{\phi} y$$

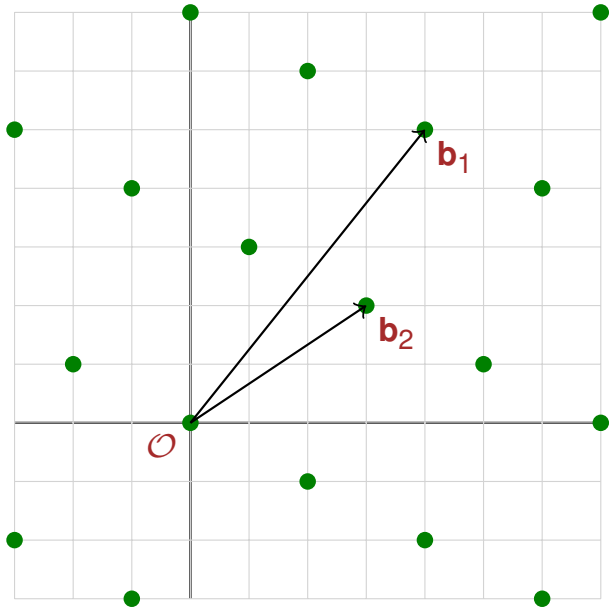
GPV Signatures, concretely

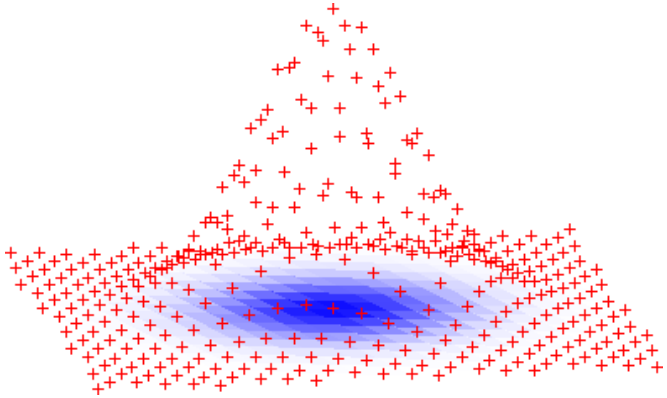
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.



GPV Signatures, concretely

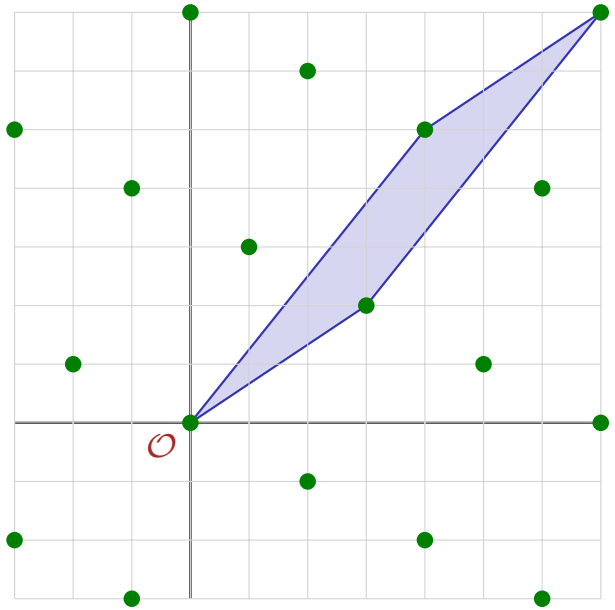
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.

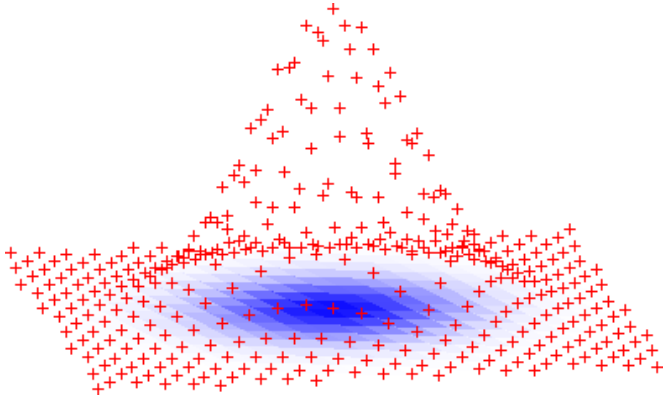




GPV Signatures, concretely

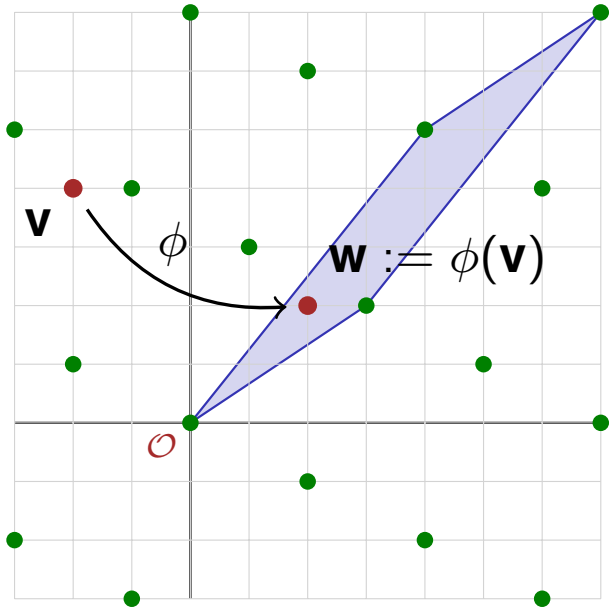
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.
- $R = \mathbb{Z}^n / \Lambda$ (fix unique representatives)

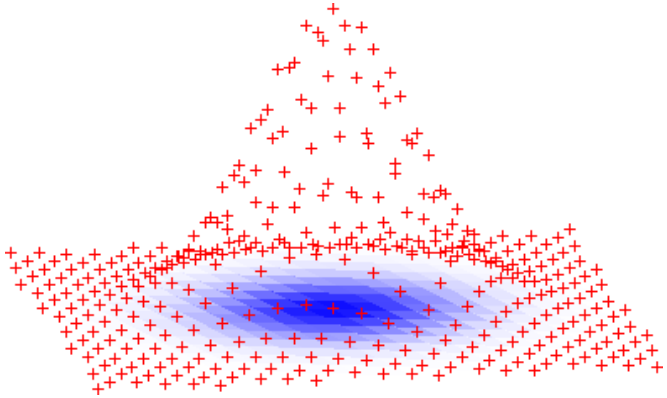




GPV Signatures, concretely

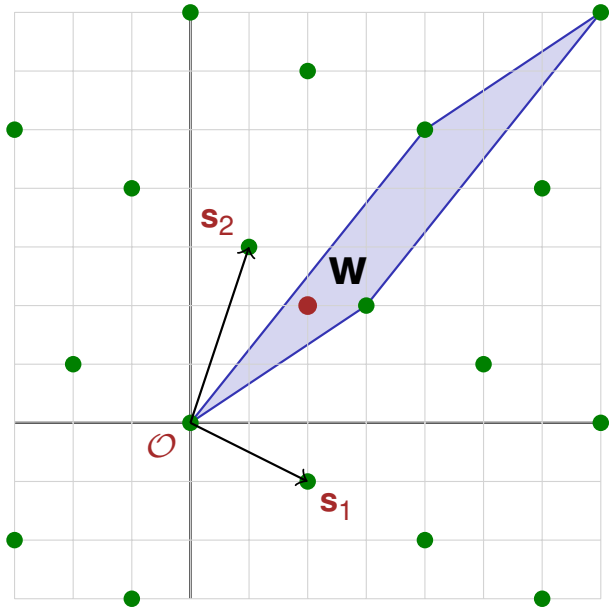
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.
- $R = \mathbb{Z}^n / \Lambda$ (fix unique representatives)
- Trapdoor function $\phi \colon \mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda)$ i.e., move $\mathbf{v}$ into a fundamental parallelepiped.

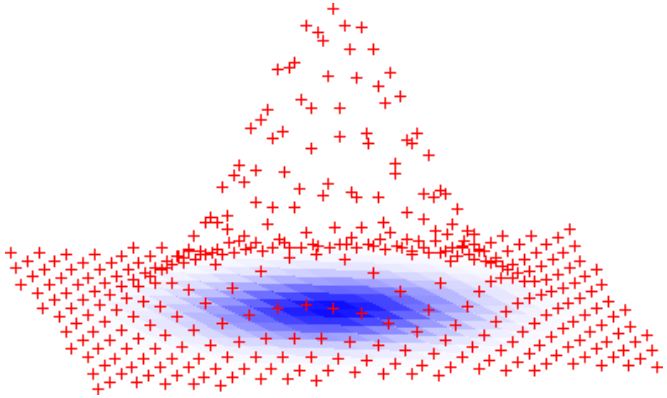




GPV Signatures, concretely

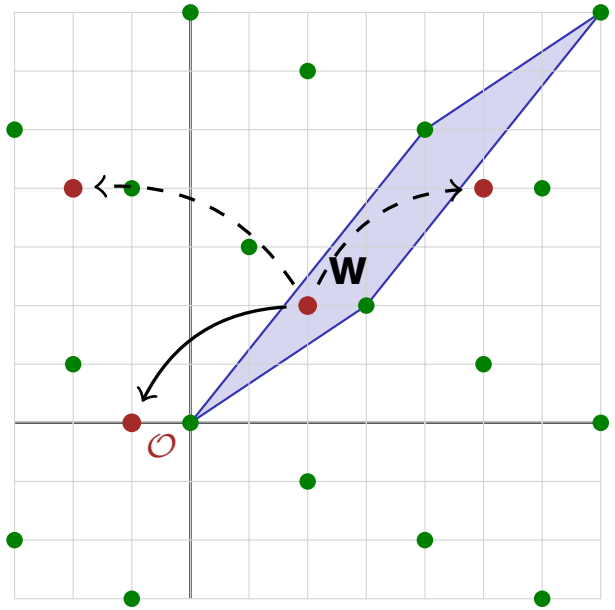
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.
- $R = \mathbb{Z}^n / \Lambda$ (fix unique representatives)
- Trapdoor function $\phi \colon \mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda)$ i.e., move $\mathbf{v}$ into a fundamental parallelepiped.
- GPV: algorithm to sample short vectors in $\Lambda + \mathbf{w}$ given a "short" basis of Λ.

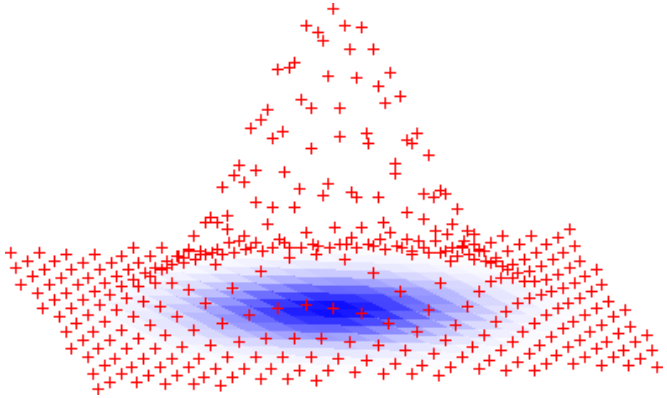




GPV Signatures, concretely

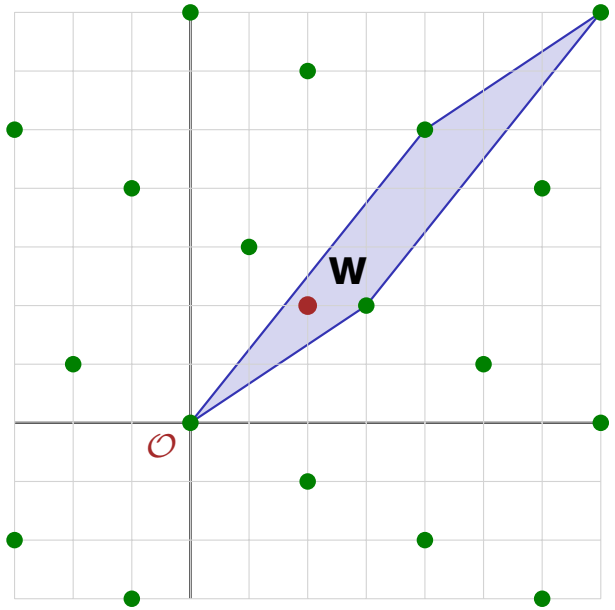
- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.
- $R = \mathbb{Z}^n / \Lambda$ (fix unique representatives)
- Trapdoor function $\phi \colon \mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda)$ i.e., move $\mathbf{v}$ into a fundamental parallelepiped.
- GPV: algorithm to sample short vectors in $\Lambda + \mathbf{w}$ given a "short" basis of Λ.

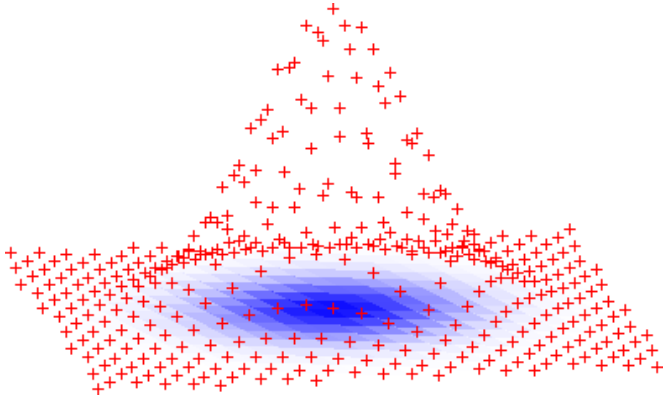




GPV Signatures, concretely

- $\Lambda \subset \mathbb{Z}^n$ a lattice (full-rank additive subgroup), defined by basis.
- $D =$ short vectors in $\mathbb{Z}^n$, with Gaussian distribution.
- $R = \mathbb{Z}^n / \Lambda$ (fix unique representatives)
- Trapdoor function $\phi \colon \mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda)$ i.e., move $\mathbf{v}$ into a fundamental parallelepiped.
- GPV: algorithm to sample short vectors in $\Lambda + \mathbf{w}$ given a "short" basis of Λ.
- Sampling from $\Lambda + \mathbf{w}$ without short basis is hard. (How hard depends on Gaussian parameter.)





Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$,
let the message space be R itself.

Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$, let the message space be *R* itself.

GPV sign/verify algorithms: $H\colon \{0,1\} \to \mathbb{Z}^n/\Lambda$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } \Lambda + H(m) \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad \sigma \bmod \Lambda = H(m) \end{aligned}$$

Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$,
let the message space be R itself.

New sign/verify algorithms: $m \in \mathbb{Z}^n/\Lambda$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } \Lambda + m \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad \sigma \bmod \Lambda = m \end{aligned}$$

Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$,
let the message space be R itself.

New sign/verify algorithms: $m \in \mathbb{Z}^n/\Lambda$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } \Lambda + m \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad \sigma \bmod \Lambda = m \end{aligned}$$

Homomorphic property: ϕ is a linear map, so adding signatures corresponds to adding messages.

Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$,
let the message space be R itself.

New sign/verify algorithms: $m \in \mathbb{Z}^n/\Lambda$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } \Lambda + m \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad \sigma \bmod \Lambda = m \end{aligned}$$

Homomorphic property: ϕ is a linear map, so adding signatures corresponds to adding messages.

- Suppose σ_1, σ_2 are signatures on m_1, m_2
  $\Rightarrow \sigma_i$ short, $\sigma_i \bmod \Lambda = m_i$.

Linearly Homomorphic Signatures: Key Idea #1

Idea: instead of hashing the messages to $R = \mathbb{Z}^n/\Lambda$,
let the message space be R itself.

New sign/verify algorithms: $m \in \mathbb{Z}^n/\Lambda$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } \Lambda + m \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad \sigma \bmod \Lambda = m \end{aligned}$$

Homomorphic property: ϕ is a linear map, so adding signatures corresponds to adding messages.

- Suppose σ_1, σ_2 are signatures on m_1, m_2
  $\Rightarrow \sigma_i$ short, $\sigma_i \bmod \Lambda = m_i$.
- For $a, b \in \mathbb{Z}$, define signature on $am_1 + bm_2$ to be
  $$\sigma := a\sigma_1 + b\sigma_2.$$
  $\Rightarrow \sigma$ is short (if a, b small), $\sigma \bmod \Lambda = am_1 + bm_2$.

Problem: Removing hash function destroys security

Valid signature doesn't imply function was computed correctly.

Problem: Removing hash function destroys security

Valid signature doesn't imply function was computed correctly.

signed grades →

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

← mean?

Problem: Removing hash function destroys security

Valid signature doesn't imply function was computed correctly.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow{87.3,\ \sigma}$

- Honest DB outputs $87.3 = \frac{1}{k}\sum s_i$ and signature $\sigma = \frac{1}{k}\sum \sigma_i$.

Problem: Removing hash function destroys security

Valid signature doesn't imply function was computed correctly.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow{\quad}$ 18.0, σ

- Honest DB outputs $87.3 = \frac{1}{k}\sum s_i$ and signature $\sigma = \frac{1}{k}\sum \sigma_i$.
- Malicious DB outputs $18.0 = s_1 - s_2$ and signature $\sigma = \sigma_1 - \sigma_2$.

Problem: Removing hash function destroys security

Valid signature doesn't imply function was computed correctly.

$\xrightarrow[\text{grades}]{\text{signed}}$

Untrusted DB

Student	Score	Sig
1	91	σ_1
2	73	σ_2
$\vdots$	$\vdots$	$\vdots$
k	84	σ_k

$\xleftarrow{\text{mean?}}$

$\xrightarrow{\quad}$ 18.0, σ

- Honest DB outputs $87.3 = \frac{1}{k}\sum s_i$ and signature $\sigma = \frac{1}{k}\sum \sigma_i$.
- Malicious DB outputs $18.0 = s_1 - s_2$ and signature $\sigma = \sigma_1 - \sigma_2$.
- σ authenticates 18, but 18 is not the mean!

Linearly Homomorphic Signatures: Key Idea #2

Use a second lattice to authenticate functions:

- $\Lambda_2 \subset \mathbb{Z}^n$ distinct from $\Lambda_1 := \Lambda$.
  - require $\Lambda_1 + \Lambda_2 = \mathbb{Z}^n$
- Map $\phi_2 \colon \mathbb{Z}^n \to \mathbb{Z}^n / \Lambda_2$ given by $\phi_2(\mathbf{v}) := \mathbf{v} \bmod \Lambda_2$.

Linearly Homomorphic Signatures: Key Idea #2

Use a second lattice to authenticate functions:

- $\Lambda_2 \subset \mathbb{Z}^n$ distinct from $\Lambda_1 := \Lambda$.
  - require $\Lambda_1 + \Lambda_2 = \mathbb{Z}^n$
- Map $\phi_2 \colon \mathbb{Z}^n \to \mathbb{Z}^n/\Lambda_2$ given by $\phi_2(\mathbf{v}) := \mathbf{v} \bmod \Lambda_2$.

Encode functions f as elements $\langle f \rangle \in \mathbb{Z}^n/\Lambda_2$.
Sign functions by computing

$$\mathrm{Sign}(f) := \text{short vector in } \Lambda_2 + \langle f \rangle.$$

Linearly Homomorphic Signatures: Key Idea #2

Use a second lattice to authenticate functions:

- $\Lambda_2 \subset \mathbb{Z}^n$ distinct from $\Lambda_1 := \Lambda$.
  - require $\Lambda_1 + \Lambda_2 = \mathbb{Z}^n$
- Map $\phi_2 \colon \mathbb{Z}^n \to \mathbb{Z}^n/\Lambda_2$ given by $\phi_2(\mathbf{v}) := \mathbf{v} \bmod \Lambda_2$.

Encode functions f as elements $\langle f \rangle \in \mathbb{Z}^n/\Lambda_2$.
Sign functions by computing

$$\mathrm{Sign}(f) := \text{short vector in } \Lambda_2 + \langle f \rangle.$$

If encoding $\langle \cdot \rangle$ is linear, (i.e., $\langle f \rangle + \langle g \rangle = \langle f + g \rangle$)
then signature is a linear operator on the space of functions.

How do we encode functions?

$k :=$ number of messages input to a function.

$$\left\{ \begin{array}{c} \text{Linear functions in} \\ k \text{ variables} \end{array} \right\} \quad \text{generated by} \quad \left\{ \begin{array}{c} k \text{ projection} \\ \text{functions } x_i \end{array} \right\}$$

- $\tau :=$ "tag" used to tie together messages in same data set.
  - prevents mixing of data from different sets

How do we encode functions?

$k :=$ number of messages input to a function.

$$\left\{ \begin{array}{c} \text{Linear functions in} \\ k \text{ variables} \end{array} \right\} \quad \text{generated by} \quad \left\{ \begin{array}{c} k \text{ projection} \\ \text{functions } x_i \end{array} \right\}$$

- $\tau :=$ "tag" used to tie together messages in same data set.
  - prevents mixing of data from different sets
- Hash function $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k$ maps $\tau \mapsto (\alpha_1, \ldots, \alpha_k)$.

How do we encode functions?

$k :=$ number of messages input to a function.

$$\left\{ \begin{array}{c} \text{Linear functions in} \\ k \text{ variables} \end{array} \right\} \quad \text{generated by} \quad \left\{ \begin{array}{c} k \text{ projection} \\ \text{functions } x_i \end{array} \right\}$$

- $\tau :=$ "tag" used to tie together messages in same data set.
  - prevents mixing of data from different sets
- Hash function $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k$ maps $\tau \mapsto (\alpha_1, \ldots, \alpha_k)$.
- Encode projection function x_i as $\alpha_i \in \mathbb{Z}^n/\Lambda_2$.

How do we encode functions?

$k :=$ number of messages input to a function.

$$\left\{ \begin{array}{c} \text{Linear functions in} \\ k \text{ variables} \end{array} \right\} \quad \text{generated by} \quad \left\{ \begin{array}{c} k \text{ projection} \\ \text{functions } x_i \end{array} \right\}$$

- $\tau :=$ "tag" used to tie together messages in same data set.
  - prevents mixing of data from different sets
- Hash function $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k$ maps $\tau \mapsto (\alpha_1, \ldots, \alpha_k)$.
- Encode projection function x_i as $\alpha_i \in \mathbb{Z}^n/\Lambda_2$.
- More generally, linear function $f = \sum c_i x_i$ on k messages encoded as $\langle f \rangle := \sum c_i \alpha_i \in \mathbb{Z}^n/\Lambda_2$.

How do we encode functions?

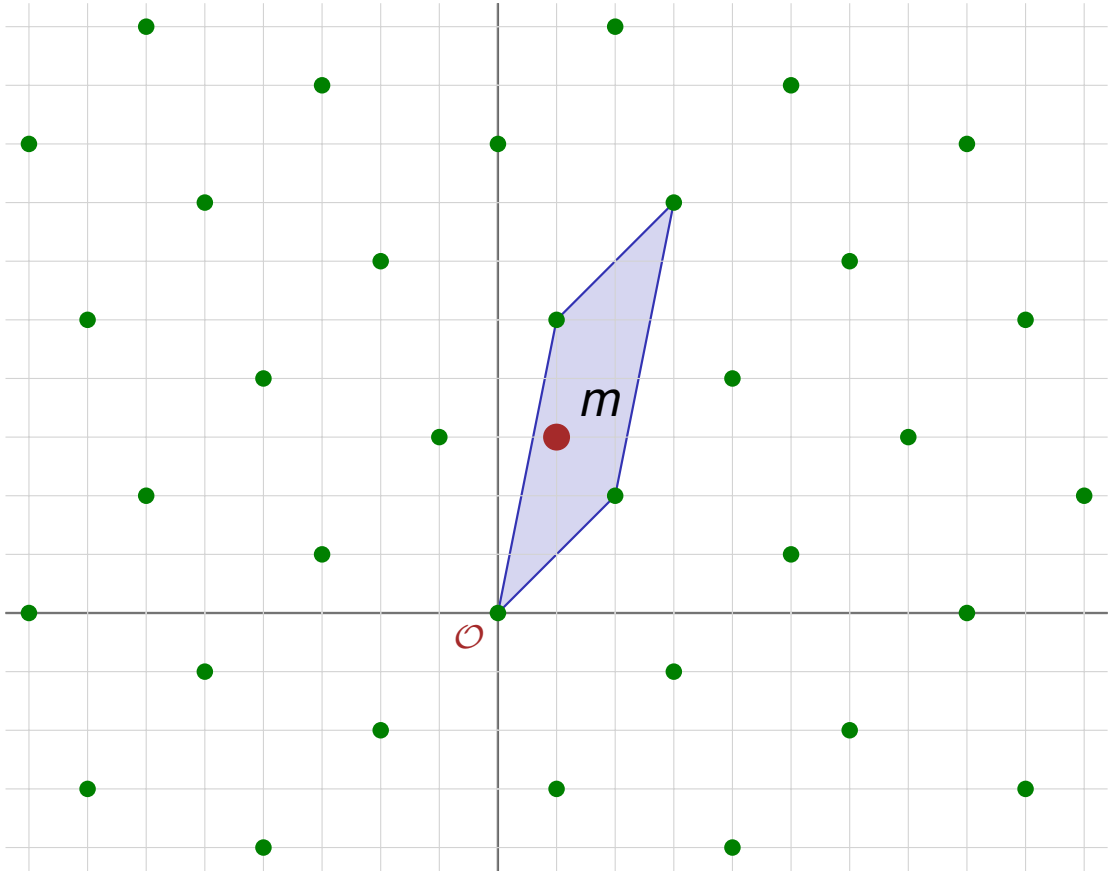
$k :=$ number of messages input to a function.

$$\left\{ \begin{array}{c} \text{Linear functions in} \\ k \text{ variables} \end{array} \right\} \quad \text{generated by} \quad \left\{ \begin{array}{c} k \text{ projection} \\ \text{functions } x_i \end{array} \right\}$$

- $\tau :=$ "tag" used to tie together messages in same data set.
  - prevents mixing of data from different sets
- Hash function $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k$ maps $\tau \mapsto (\alpha_1, \ldots, \alpha_k)$.
- Encode projection function x_i as $\alpha_i \in \mathbb{Z}^n/\Lambda_2$.
- More generally, linear function $f = \sum c_i x_i$ on k messages encoded as $\langle f \rangle := \sum c_i \alpha_i \in \mathbb{Z}^n/\Lambda_2$.
  - Encoding is one-way.
  - Sweeping under the rug (for now): where do the c_i live? (think of c_i as small integers)

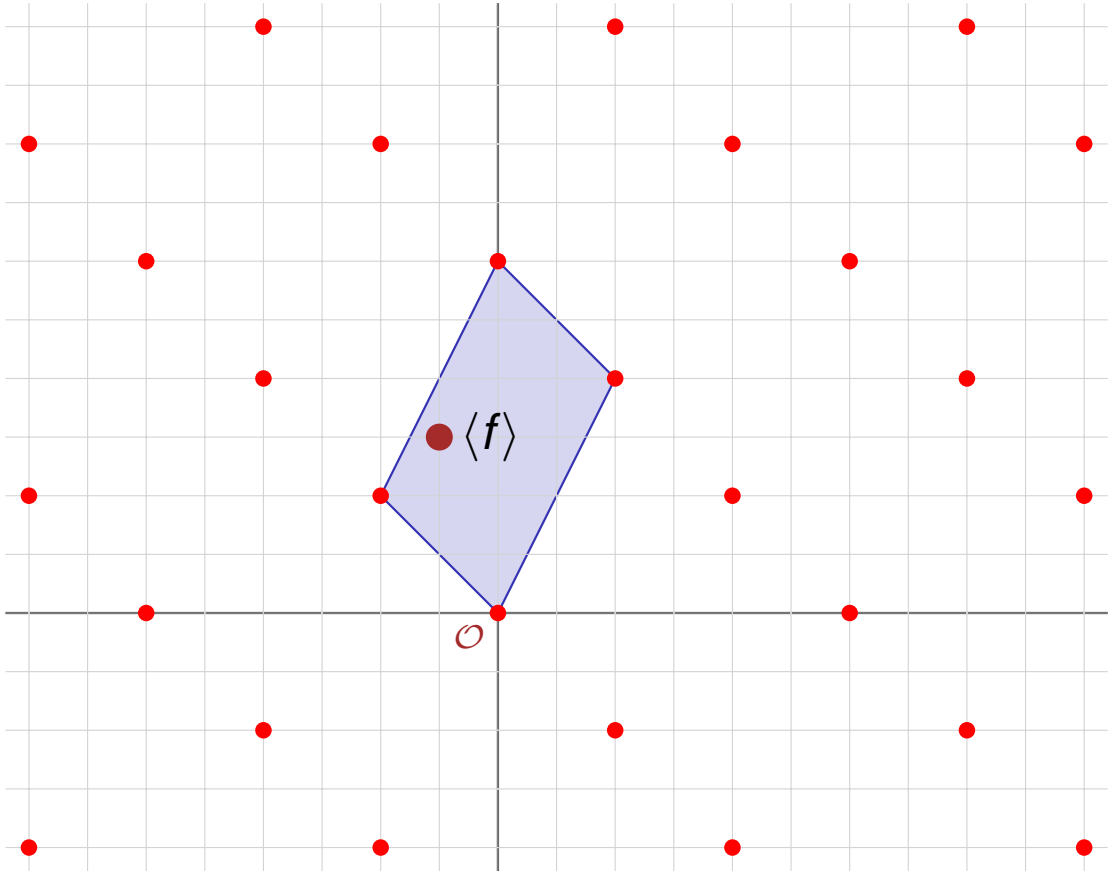
“Intersection method” binds messages to functions

- Messages $m \in \mathbb{Z}^n/\Lambda_1$.



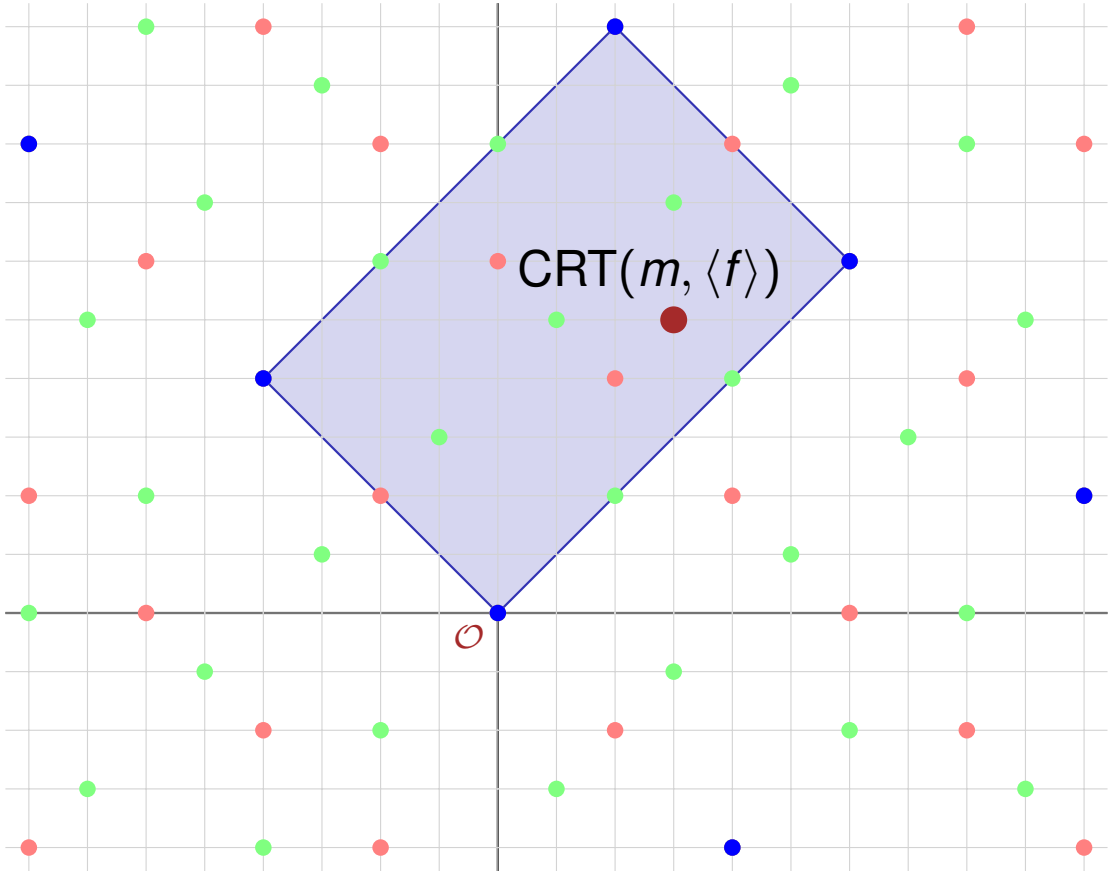
“Intersection method” binds messages to functions

- Messages $m \in \mathbb{Z}^n/\Lambda_1$.
- Functions $f = \sum c_i x_i$ encoded as $\langle f \rangle = \sum c_i \alpha_i \in \mathbb{Z}^n/\Lambda_2$.
  (α_i defined by tag τ.)



“Intersection method” binds messages to functions

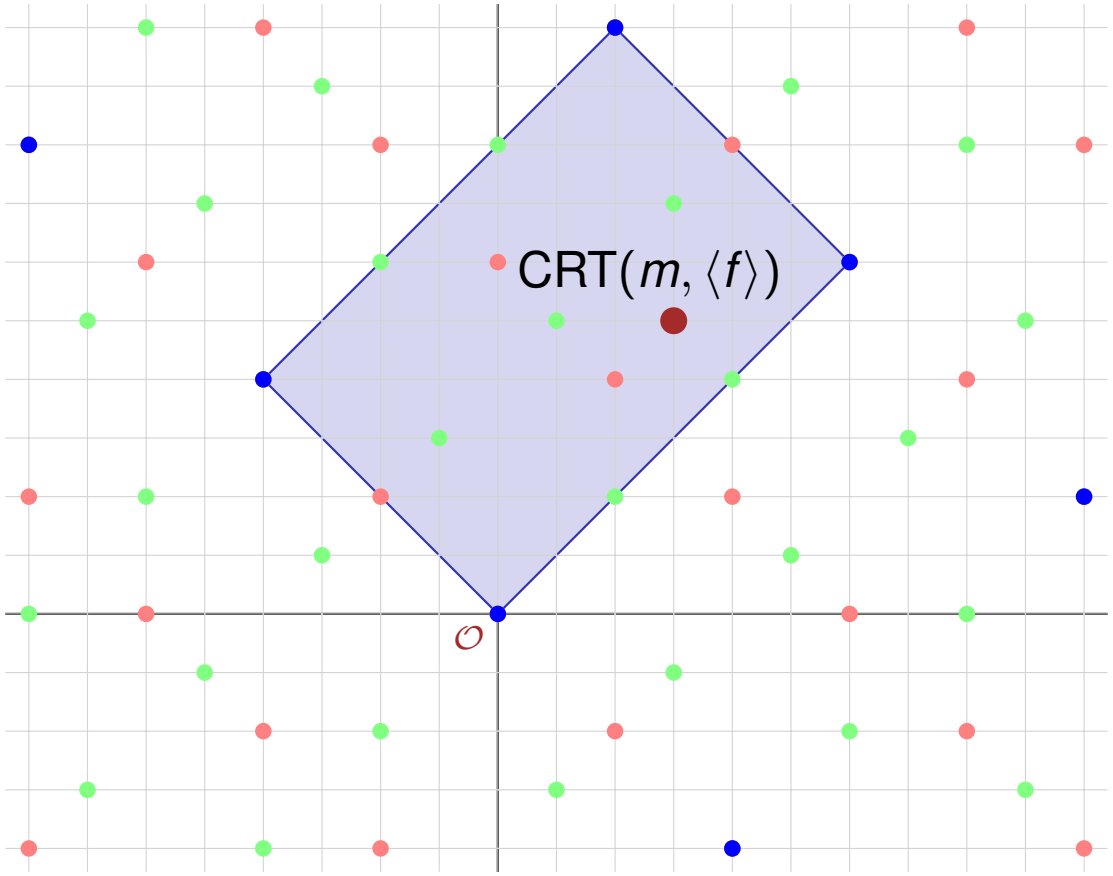
- Messages $m \in \mathbb{Z}^n/\Lambda_1$.
- Functions $f = \sum c_i x_i$ encoded as $\langle f \rangle = \sum c_i \alpha_i \in \mathbb{Z}^n/\Lambda_2$.
  (α_i defined by tag τ.)
- Pair $(m, \langle f \rangle)$ gives unique element of $\mathbb{Z}^n/\Lambda_1 \cap \Lambda_2$.



$\mathrm{CRT}(m, \langle f \rangle)$
$= m \bmod \Lambda_1$
$= \langle f \rangle \bmod \Lambda_2$

Signing a message-function pair

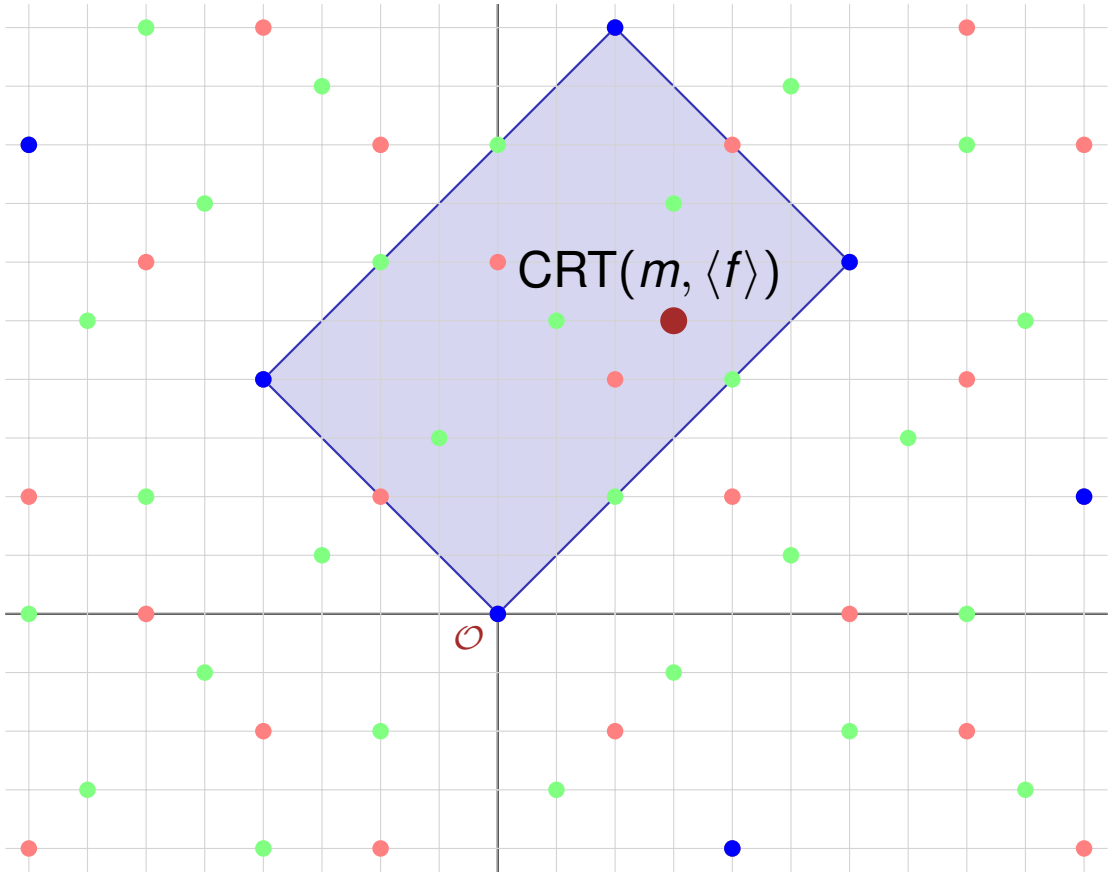
- Pair $(m, \langle f \rangle)$ gives unique element of $\mathbb{Z}^n / \Lambda_1 \cap \Lambda_2$.



$\mathrm{CRT}(m, \langle f \rangle)$
$= m \bmod \Lambda_1$
$= \langle f \rangle \bmod \Lambda_2$

Signing a message-function pair

- Pair $(m, \langle f \rangle)$ gives unique element of $\mathbb{Z}^n / \Lambda_1 \cap \Lambda_2$.



CRT$(m, \langle f \rangle)$
$= m \bmod \Lambda_1$
$= \langle f \rangle \bmod \Lambda_2$

$$\begin{aligned} \text{Sign}(m) &:= \text{short vector in } (\Lambda_1 \cap \Lambda_2) + \text{CRT}(m, \langle f \rangle) \\ \text{Verify}(\sigma) &:= 1 \quad \text{iff} \quad (\sigma \bmod \Lambda_1 = m) \text{ and } (\sigma \bmod \Lambda_2 = \langle f \rangle) \end{aligned}$$

Linearly homomorphic signature scheme

- KeyGen(n):
  - $pk =$ Lattices $\Lambda_1, \Lambda_2 \subset \mathbb{Z}^n$, Gaussian parameter β
  - $sk =$ short basis of $\Lambda_1 \cap \Lambda_2$
  - $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.

Linearly homomorphic signature scheme

- KeyGen(n):
  - $pk =$ Lattices $\Lambda_1, \Lambda_2 \subset \mathbb{Z}^n$, Gaussian parameter β
  - $sk =$ short basis of $\Lambda_1 \cap \Lambda_2$
  - $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i): compute short vector σ_i in $\Lambda_1 \cap \Lambda_2 + \mathrm{CRT}(m_i, \alpha_i)$.
  - $x_i =$ ith projection function
  - How short? $\|\sigma\| < \beta\sqrt{n}$

Linearly homomorphic signature scheme

- KeyGen(n):
  - $pk =$ Lattices $\Lambda_1, \Lambda_2 \subset \mathbb{Z}^n$, Gaussian parameter β
  - $sk =$ short basis of $\Lambda_1 \cap \Lambda_2$
  - $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i): compute short vector σ_i in $\Lambda_1 \cap \Lambda_2 + \mathrm{CRT}(m_i, \alpha_i)$.
  - $x_i =$ ith projection function
  - How short? $\|\sigma\| < \beta\sqrt{n}$
- Evaluate($f = \sum c_i x_i, (\sigma_1, \ldots, \sigma_k)$): compute $\sigma = \sum c_i \sigma_i$.
  - lift c_i defined mod Λ_1 to $c_i \in \mathbb{Z}$.

Linearly homomorphic signature scheme

- KeyGen(n):
  - $pk =$ Lattices $\Lambda_1, \Lambda_2 \subset \mathbb{Z}^n$, Gaussian parameter β
  - $sk =$ short basis of $\Lambda_1 \cap \Lambda_2$
  - $H\colon \{0,1\}^* \to (\mathbb{Z}^n/\Lambda_2)^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i): compute short vector σ_i in $\Lambda_1 \cap \Lambda_2 + \mathrm{CRT}(m_i, \alpha_i)$.
  - $x_i =$ ith projection function
  - How short? $\|\sigma\| < \beta\sqrt{n}$
- Evaluate($f = \sum c_i x_i, (\sigma_1, \ldots, \sigma_k)$): compute $\sigma = \sum c_i \sigma_i$.
  - lift c_i defined mod Λ_1 to $c_i \in \mathbb{Z}$.
- Verify($\tau, \sigma, m, f = \sum c_i x_i$): Accept if
  1. σ mod $\Lambda_1 = m$,
  2. σ mod $\Lambda_2 = \langle f \rangle = \sum c_i \alpha_i$,
  3. σ sufficiently short: $\|\sigma\| < \beta\sqrt{n} \cdot k \cdot \max\{|\mathrm{lift}(c_i)|\}$.

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$

  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$

  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].
- $\Lambda_1 \cap \Lambda_2 = p \cdot \Lambda_q^{\perp}(\mathbf{A})$; short basis is $p \cdot \mathbf{B}$.

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$

  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].
- $\Lambda_1 \cap \Lambda_2 = p \cdot \Lambda_q^{\perp}(\mathbf{A})$; short basis is $p \cdot \mathbf{B}$.
- Message space: $\mathbb{Z}^n / p\mathbb{Z}^n = \mathbb{F}_p^n$.

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$
  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].
- $\Lambda_1 \cap \Lambda_2 = p \cdot \Lambda_q^{\perp}(\mathbf{A})$; short basis is $p \cdot \mathbf{B}$.
- Message space: $\mathbb{Z}^n / p\mathbb{Z}^n = \mathbb{F}_p^n$.
- Admissible functions $f = \sum c_i x_i$, $c_i \in \mathbb{F}_p$:
  $\mathbb{F}_p$-linear combinations of k vectors in $\mathbb{F}_p^n$.
  - lift c_i to $\mathbb{Z}$ by mapping to preimage in $(-p/2, p/2]$.

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$
  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].
- $\Lambda_1 \cap \Lambda_2 = p \cdot \Lambda_q^{\perp}(\mathbf{A})$; short basis is $p \cdot \mathbf{B}$.
- Message space: $\mathbb{Z}^n / p\mathbb{Z}^n = \mathbb{F}_p^n$.
- Admissible functions $f = \sum c_i x_i$, $c_i \in \mathbb{F}_p$:
  $\mathbb{F}_p$-linear combinations of k vectors in $\mathbb{F}_p^n$.
  - lift c_i to $\mathbb{Z}$ by mapping to preimage in $(-p/2, p/2]$.

Signature scheme signs k vectors $\mathbf{v}_i \in \mathbb{F}_p^n$ and can authenticate any $\mathbb{F}_p$-linear combination of the $\mathbf{v}_i$.

Concrete example

- $\Lambda_1 = p\mathbb{Z}^n$, p small prime
- $\Lambda_2 = \Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{A} \cdot \mathbf{x} = 0 \bmod q\}$
  ($q \neq p$ prime, $\mathbf{A} \in \mathbb{F}_q^{n' \times n}$)
  - Can sample random $\Lambda_q^{\perp}(\mathbf{A})$ with short basis $\mathbf{B}$ [A99,AP09].
- $\Lambda_1 \cap \Lambda_2 = p \cdot \Lambda_q^{\perp}(\mathbf{A})$; short basis is $p \cdot \mathbf{B}$.
- Message space: $\mathbb{Z}^n / p\mathbb{Z}^n = \mathbb{F}_p^n$.
- Admissible functions $f = \sum c_i x_i$, $c_i \in \mathbb{F}_p$:
  $\mathbb{F}_p$-linear combinations of k vectors in $\mathbb{F}_p^n$.
  - lift c_i to $\mathbb{Z}$ by mapping to preimage in $(-p/2, p/2]$.

Signature scheme signs k vectors $\mathbf{v}_i \in \mathbb{F}_p^n$ and can authenticate any $\mathbb{F}_p$-linear combination of the $\mathbf{v}_i$.

Same functionality as *network coding* signatures [BFKW09,GKKR10], except p can be small (even $p = 2$).

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall. $\xrightarrow{\quad pk \quad}$ Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

$\xrightarrow{\quad pk \quad}$

$\xleftarrow{\quad \text{data } m_1, \ldots, m_k \quad}$

Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

$\xrightarrow{\quad pk \quad}$

$\xleftarrow{\quad \text{data } m_1, \ldots, m_k \quad}$

$\xrightarrow{\quad \text{tag } \tau,\ \text{sigs } \sigma_1, \ldots, \sigma_k \quad}$

Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

$\xrightarrow{\quad pk \quad}$

$\xleftarrow{\quad \text{data } m_1, \ldots, m_k \quad}$

$\xrightarrow{\quad \text{tag } \tau,\ \text{sigs } \sigma_1, \ldots, \sigma_k \quad}$

$\Big\}$ repeat

Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

$\xrightarrow{\quad pk \quad}$

$\xleftarrow{\quad \text{data } m_1, \ldots, m_k \quad}$

$\xrightarrow{\quad \text{tag } \tau, \text{ sigs } \sigma_1, \ldots, \sigma_k \quad}$

$\xleftarrow{\quad \text{forgery } \tau^*, m^*, \sigma^*, f \quad}$

repeat

Adversary

Security

What does it mean to forge a homomorphic signature?

- Forgery is a valid signature on (τ, m^*, f) with $m^* \neq f(\text{messages with tag } \tau)$.

Chall.

$\xrightarrow{\quad pk \quad}$

$\xleftarrow{\quad \text{data } m_1, \ldots, m_k \quad}$

$\xrightarrow{\quad \text{tag } \tau, \text{ sigs } \sigma_1, \ldots, \sigma_k \quad}$

$\xleftarrow{\quad \text{forgery } \tau^*, m^*, \sigma^*, f \quad}$

repeat

Adversary

Adversary wins if f admissible, σ^* verifies for (τ^*, m^*, f), and

1. τ^* not obtained in response to a query, or
2. $\tau^* = \tau$ for query $(m_1, \ldots, m_k)$, and $m^* \neq f(m_1, \ldots, m_k)$.

Security Theorem

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero vector in Λ_2.

Security Theorem

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero vector in Λ_2.

To implement system: choose Λ_2 such that finding short vectors in Λ_2 is hard!

- $\Lambda_q^{\perp}(\mathbf{A})$ has this property [MR04,GPV08].

Security Theorem

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero vector in Λ_2.

To implement system: choose Λ_2 such that finding short vectors in Λ_2 is hard!

- $\Lambda_q^{\perp}(\mathbf{A})$ has this property [MR04,GPV08].

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.
2. Use adversary's forgery to produce a short vector in Λ_2.

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

- Given challenge Λ_2, generate Λ_1 with a short basis.

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

- Given challenge Λ_2, generate Λ_1 with a short basis.
- Adversary queries $m_1, \ldots, m_k$.

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

- Given challenge Λ_2, generate Λ_1 with a short basis.
- Adversary queries $m_1, \ldots, m_k$.
- Choose random τ and simulate signature σ_i on (τ, m_i, x_i):
  1. Use basis of Λ_1 to compute short vectors $\sigma_i \in \Lambda_1 + m_i$;
  2. Set $\alpha_i := \sigma_i \bmod \Lambda_2 \in \mathbb{Z}^n/\Lambda_2$.
  3. Program random oracle with $H(\tau) := (\alpha_1, \ldots, \alpha_k)$.

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

- Given challenge Λ_2, generate Λ_1 with a short basis.
- Adversary queries $m_1, \ldots, m_k$.
- Choose random τ and simulate signature σ_i on (τ, m_i, x_i):
  1. Use basis of Λ_1 to compute short vectors $\sigma_i \in \Lambda_1 + m_i$;
  2. Set $\alpha_i := \sigma_i \bmod \Lambda_2 \in \mathbb{Z}^n/\Lambda_2$.
  3. Program random oracle with $H(\tau) := (\alpha_1, \ldots, \alpha_k)$.
- For certain parameter choices, α_i are statistically close to uniform in $\mathbb{Z}^n/\Lambda_2$

Security Proof, Part 1

Proof outline

1. Answer signature queries without a basis of $\Lambda_1 \cap \Lambda_2$.

- Given challenge Λ_2, generate Λ_1 with a short basis.
- Adversary queries $m_1, \ldots, m_k$.
- Choose random τ and simulate signature σ_i on (τ, m_i, x_i):
  1. Use basis of Λ_1 to compute short vectors $\sigma_i \in \Lambda_1 + m_i$;
  2. Set $\alpha_i := \sigma_i \bmod \Lambda_2 \in \mathbb{Z}^n/\Lambda_2$.
  3. Program random oracle with $H(\tau) := (\alpha_1, \ldots, \alpha_k)$.
- For certain parameter choices, α_i are statistically close to uniform in $\mathbb{Z}^n/\Lambda_2$
- Simulation is indistinguishable from real system.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.
- Compute $\sigma := \sum c_i \sigma_i =$ "real" sig on $f(m_1, \ldots, m_k)$.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.
- Compute $\sigma := \sum c_i \sigma_i =$ "real" sig on $f(m_1, \ldots, m_k)$.
- Validity of forged σ^* means:
  1. $\sigma^* \bmod \Lambda_1 = m^* \neq f(m_1, \ldots, m_k)$
  2. $\sigma^* \bmod \Lambda_2 = \sum c_i \alpha_i$
  3. σ^* is short

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.
- Compute $\sigma := \sum c_i \sigma_i =$ "real" sig on $f(m_1, \ldots, m_k)$.
- Validity of forged σ^* and authenticity of "real" σ means:
  1. $\sigma^* \bmod \Lambda_1 = m^* \neq f(m_1, \ldots, m_k) = \sigma \bmod \Lambda_1$
  2. $\sigma^* \bmod \Lambda_2 = \sum c_i \alpha_i = \sigma \bmod \Lambda_2$
  3. σ^* is short and σ is short

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.
- Compute $\sigma := \sum c_i \sigma_i =$ "real" sig on $f(m_1, \ldots, m_k)$.
- Validity of forged σ^* and authenticity of "real" σ means:
  1. $\sigma^* \bmod \Lambda_1 = m^* \neq f(m_1, \ldots, m_k) = \sigma \bmod \Lambda_1$
  2. $\sigma^* \bmod \Lambda_2 = \sum c_i \alpha_i = \sigma \bmod \Lambda_2$
  3. σ^* is short and σ is short

Conclusion: $\sigma^* - \sigma$ is (1) nonzero, (2) in Λ_2, (3) short.

Security Proof, Part 2

Proof outline

2. Use forgery to produce a short nonzero vector in Λ_2.

- Adversary outputs forgery $(\tau^*, m^*, \sigma^*, f = \sum c_i x_i)$.
- Suppose $\tau^*, \sigma_1, \ldots, \sigma_k$ answer query $m_1, \ldots, m_k$.
- Compute $\sigma := \sum c_i \sigma_i =$ "real" sig on $f(m_1, \ldots, m_k)$.
- Validity of forged σ^* and authenticity of "real" σ means:
  1. $\sigma^* \bmod \Lambda_1 = m^* \neq f(m_1, \ldots, m_k) = \sigma \bmod \Lambda_1$
  2. $\sigma^* \bmod \Lambda_2 = \sum c_i \alpha_i = \sigma \bmod \Lambda_2$
  3. σ^* is short and σ is short

Conclusion: $\sigma^* - \sigma$ is (1) nonzero, (2) in Λ_2, (3) short.

- If τ^* not obtained from a query, sign random messages m_i and perform same analysis.

Privacy

Privacy property: derived signature on $f(m_1, \ldots, m_k)$ reveals nothing about $m_1, \ldots, m_k$ beyond value of f.

Privacy

Privacy property: derived signature on $f(m_1, \ldots, m_k)$ reveals nothing about $m_1, \ldots, m_k$ beyond value of f.

Specifically: given data sets

$$\vec{m} = (m_1, \ldots, m_k), \qquad \vec{m}' = (m'_1, \ldots, m'_k)$$

and admissible function f with

$$f(\vec{m}) = f(\vec{m}'),$$

even unbounded adversary cannot distinguish derived signature on $f(\vec{m})$ from derived signature on $f(\vec{m}')$.

Privacy theorem

Theorem

Linearly homomorphic signatures are private.

Privacy theorem

Theorem

Linearly homomorphic signatures are private.

Proof idea

- Distribution of derived signature on $f(\vec{m})$ depends only on f and $f(\vec{m})$, not on $\vec{m}$.
- If $f(\vec{m}) = f(\vec{m}')$, distributions of derived sigs are identical.

Privacy theorem

Theorem

Linearly homomorphic signatures are private.

Proof idea

- Distribution of derived signature on $f(\vec{m})$ depends only on f and $f(\vec{m})$, not on $\vec{m}$.
- If $f(\vec{m}) = f(\vec{m}')$, distributions of derived sigs are identical.

Key fact in proof: distribution of linear combination of Gaussian samples is also Gaussian.

- Sigs on m_i sampled from discrete Gaussian distribution.

Polynomially Homomorphic Signatures from Ideal Lattices

Extending the system

Linearly homomorphic scheme: messages in $\mathbb{Z}^n/\Lambda_1$, signatures in $\mathbb{Z}^n$.

$\phi\colon \mathbb{Z}^n \to \mathbb{Z}^n/\Lambda_1$ given by $\mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda_1)$ is a linear map, so we can add both before and after applying ϕ.

Extending the system

Linearly homomorphic scheme: messages in $\mathbb{Z}^n/\Lambda_1$, signatures in $\mathbb{Z}^n$.

$\phi\colon \mathbb{Z}^n \to \mathbb{Z}^n/\Lambda_1$ given by $\mathbf{v} \mapsto (\mathbf{v} \bmod \Lambda_1)$ is a linear map, so we can add both before and after applying ϕ.

New idea: what if $\mathbb{Z}^n$ has a ring structure and Λ_1 is an ideal?

Then ϕ is a ring homomorphism, so we can add or multiply both before and after applying ϕ.

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{rcl} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{rcl} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

- $\Lambda_1, \Lambda_2 =$ ideals $\mathfrak{p}, \mathfrak{q} \subset R$ of prime norm p, q.

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{rcl} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

- $\Lambda_1, \Lambda_2 =$ ideals $\mathfrak{p}, \mathfrak{q} \subset R$ of prime norm p, q.
- Message space is $R/\mathfrak{p} = \mathbb{F}_p$.

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{rcl} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

- $\Lambda_1, \Lambda_2 =$ ideals $\mathfrak{p}, \mathfrak{q} \subset R$ of prime norm p, q.
- Message space is $R/\mathfrak{p} = \mathbb{F}_p$.
- Admissible functions are polynomials $f \in \mathbb{F}_p[x_1, \ldots, x_k]$.

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{rcl} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

- $\Lambda_1, \Lambda_2 =$ ideals $\mathfrak{p}, \mathfrak{q} \subset R$ of prime norm p, q.
- Message space is $R/\mathfrak{p} = \mathbb{F}_p$.
- Admissible functions are polynomials $f \in \mathbb{F}_p[x_1, \ldots, x_k]$.
- Polynomials encoded in $R/\mathfrak{q} = \mathbb{F}_q$ using hash function

$$\begin{array}{rcl} H\colon \{0,1\}^* & \to & \mathbb{F}_q^k \\ \tau & \mapsto & (\alpha_1, \ldots, \alpha_k). \end{array}$$

Setup for polynomial system [G09]

Fix monic, irreducible $F(x) \in \mathbb{Z}[x]$ of degree n.

- $R := \mathbb{Z}[x]/(F(x))$ gives a ring structure on $\mathbb{Z}^n$:

$$\begin{array}{ccc} \text{(coordinates of vectors)} & \leftrightarrow & \text{(coefficients of polynomials mod } F\text{)} \\ (a_0, \ldots, a_{n-1}) & \leftrightarrow & a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} \end{array}$$

- $\Lambda_1, \Lambda_2 =$ ideals $\mathfrak{p}, \mathfrak{q} \subset R$ of prime norm p, q.
- Message space is $R/\mathfrak{p} = \mathbb{F}_p$.
- Admissible functions are polynomials $f \in \mathbb{F}_p[x_1, \ldots, x_k]$.
- Polynomials encoded in $R/\mathfrak{q} = \mathbb{F}_q$ using hash function

$$\begin{array}{rcl} H\colon \{0,1\}^* & \to & \mathbb{F}_q^k \\ \tau & \mapsto & (\alpha_1, \ldots, \alpha_k). \end{array}$$

- Encoding uses lift of f to $\hat{f} \in \mathbb{Z}[x_1, \ldots, x_k]$:
  f encoded as $\hat{f}(\alpha_1, \ldots, \alpha_k)$ mod q.

The polynomial system, concretely

- KeyGen(n):
  - $F(x) \in \mathbb{Z}[x]$ degree n
    $\Rightarrow$ ring structure on $\mathbb{Z}^n \cong R := \mathbb{Z}[x]/(F(x))$.
  - $pk =$ prime ideals $\mathfrak{p}, \mathfrak{q} \subset R$, Gaussian parameter β
  - $sk =$ short basis of $\mathfrak{p} \cap \mathfrak{q} = \mathfrak{p} \cdot \mathfrak{q}$
  - $H\colon \{0,1\}^* \to \mathbb{F}_q^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.

The polynomial system, concretely

- KeyGen(n):
  - $F(x) \in \mathbb{Z}[x]$ degree n
    $\Rightarrow$ ring structure on $\mathbb{Z}^n \cong R := \mathbb{Z}[x]/(F(x))$.
  - $pk =$ prime ideals $\mathfrak{p}, \mathfrak{q} \subset R$, Gaussian parameter β
  - $sk =$ short basis of $\mathfrak{p} \cap \mathfrak{q} = \mathfrak{p} \cdot \mathfrak{q}$
  - $H\colon \{0,1\}^* \to \mathbb{F}_q^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i):
  - Compute short element σ_i in $\mathfrak{p} \cdot \mathfrak{q} + \mathrm{CRT}(m_i, \alpha_i)$.

The polynomial system, concretely

- KeyGen(n):
  - $F(x) \in \mathbb{Z}[x]$ degree n
    $\Rightarrow$ ring structure on $\mathbb{Z}^n \cong R := \mathbb{Z}[x]/(F(x))$.
  - $pk =$ prime ideals $\mathfrak{p}, \mathfrak{q} \subset R$, Gaussian parameter β
  - $sk =$ short basis of $\mathfrak{p} \cap \mathfrak{q} = \mathfrak{p} \cdot \mathfrak{q}$
  - $H\colon \{0,1\}^* \to \mathbb{F}_q^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i):
  - Compute short element σ_i in $\mathfrak{p} \cdot \mathfrak{q} +$ CRT(m_i, α_i).
- Evaluate($f, (\sigma_1, \ldots, \sigma_k)$):
  - Lift $f \in \mathbb{F}_p[x_1, \ldots, x_k]$ to $\hat{f} \in \mathbb{Z}[x_1, \ldots, x_k]$.
  - Output $\sigma = \hat{f}(\sigma_1, \ldots, \sigma_k) \in R$

The polynomial system, concretely

- KeyGen(n):
  - $F(x) \in \mathbb{Z}[x]$ degree n
    $\Rightarrow$ ring structure on $\mathbb{Z}^n \cong R := \mathbb{Z}[x]/(F(x))$.
  - $pk =$ prime ideals $\mathfrak{p}, \mathfrak{q} \subset R$, Gaussian parameter β
  - $sk =$ short basis of $\mathfrak{p} \cap \mathfrak{q} = \mathfrak{p} \cdot \mathfrak{q}$
  - $H\colon \{0,1\}^* \to \mathbb{F}_q^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i):
  - Compute short element σ_i in $\mathfrak{p} \cdot \mathfrak{q} + \mathrm{CRT}(m_i, \alpha_i)$.
- Evaluate($f, (\sigma_1, \ldots, \sigma_k)$):
  - Lift $f \in \mathbb{F}_p[x_1, \ldots, x_k]$ to $\hat{f} \in \mathbb{Z}[x_1, \ldots, x_k]$.
  - Output $\sigma = \hat{f}(\sigma_1, \ldots, \sigma_k) \in R$
- Verify(τ, σ, m, f): Accept if
  1. $\sigma \bmod \mathfrak{p} = m$,
  2. $\sigma \bmod \mathfrak{q} = \hat{f}(\alpha_1, \ldots, \alpha_k) \bmod q$,
  3. σ sufficiently short

The polynomial system, concretely

- KeyGen(n):
  - $F(x) \in \mathbb{Z}[x]$ degree n
    $\Rightarrow$ ring structure on $\mathbb{Z}^n \cong R := \mathbb{Z}[x]/(F(x))$.
  - $pk =$ prime ideals $\mathfrak{p}, \mathfrak{q} \subset R$, Gaussian parameter β
  - $sk =$ short basis of $\mathfrak{p} \cap \mathfrak{q} = \mathfrak{p} \cdot \mathfrak{q}$ — how to generate?
  - $H\colon \{0,1\}^* \to \mathbb{F}_q^k, \quad H(\tau) = (\alpha_1, \ldots, \alpha_k)$.
- Sign(τ, m_i, x_i):
  - Compute short element σ_i in $\mathfrak{p} \cdot \mathfrak{q} + \mathrm{CRT}(m_i, \alpha_i)$.
- Evaluate($f, (\sigma_1, \ldots, \sigma_k)$):
  - Lift $f \in \mathbb{F}_p[x_1, \ldots, x_k]$ to $\hat{f} \in \mathbb{Z}[x_1, \ldots, x_k]$.
  - Output $\sigma = \hat{f}(\sigma_1, \ldots, \sigma_k) \in R$ — why is this short?
- Verify(τ, σ, m, f): Accept if
  1. $\sigma \bmod \mathfrak{p} = m$,
  2. $\sigma \bmod \mathfrak{q} = \hat{f}(\alpha_1, \ldots, \alpha_k) \bmod q$,
  3. σ sufficiently short — how short?

Products of short elements

Why is a product of short elements of R short?

Products of short elements

Why is a product of short elements of R short?

[G09,G10]: parameter γ_F measures how much multiplication in R increases length:

$$\gamma_F := \sup_{u,v \in R} \frac{\|u \cdot v\|}{\|u\| \cdot \|v\|}.$$

- Product of d elements of length $< \beta$ has length $< \gamma_F^{d-1} \beta^d$.
- If $\beta, \gamma_F \in \text{poly}(n)$ and $d = O(1)$, then this is still considered "short".

Products of short elements

Why is a product of short elements of R short?

[G09,G10]: parameter γ_F measures how much multiplication in R increases length:

$$\gamma_F := \sup_{u,v \in R} \frac{\|u \cdot v\|}{\|u\| \cdot \|v\|}.$$

- Product of d elements of length $< \beta$ has length $< \gamma_F^{d-1} \beta^d$.
- If $\beta, \gamma_F \in \text{poly}(n)$ and $d = O(1)$, then this is still considered "short".

Lots of $F(x)$ have small γ_F:

- Cyclotomic polynomials $\Phi_\ell(x)$, ℓ prime or $\ell = 2^a 3^b$.
- $F(x) = x^n - h(x)$, $\deg(h) < n/2$.
- Others...

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.
- Ideal $\mathfrak{p} = u \cdot R$ is generated (over $\mathbb{Z}$) by

$$\mathbf{B} := \{u,\ u \cdot x,\ u \cdot x^2, \ldots,\ u \cdot x^{n-1}\}.$$

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.
- Ideal $\mathfrak{p} = u \cdot R$ is generated (over $\mathbb{Z}$) by

$$\mathbf{B} := \{u,\ u \cdot x,\ u \cdot x^2, \ldots,\ u \cdot x^{n-1}\}.$$

- Each element of $\mathbf{B}$ has norm at most $\|u\| \cdot \gamma_F$
  $\Rightarrow$ $\mathbf{B}$ is a short basis of $\mathfrak{p}$.

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.
- Ideal $\mathfrak{p} = u \cdot R$ is generated (over $\mathbb{Z}$) by

$$\mathbf{B} := \{u,\ u \cdot x,\ u \cdot x^2, \ldots,\ u \cdot x^{n-1}\}.$$

- Each element of $\mathbf{B}$ has norm at most $\|u\| \cdot \gamma_F$
  $\Rightarrow$ $\mathbf{B}$ is a short basis of $\mathfrak{p}$.
- Repeat to get $\mathfrak{q}$ with short generator w.

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.
- Ideal $\mathfrak{p} = u \cdot R$ is generated (over $\mathbb{Z}$) by

$$\mathbf{B} := \{u,\ u \cdot x,\ u \cdot x^2, \ldots,\ u \cdot x^{n-1}\}.$$

- Each element of $\mathbf{B}$ has norm at most $\|u\| \cdot \gamma_F$
  $\Rightarrow$ $\mathbf{B}$ is a short basis of $\mathfrak{p}$.
- Repeat to get $\mathfrak{q}$ with short generator w.
- $\{u \cdot w \cdot x^i\}_{i=0}^{n-1}$ is a short basis of $\mathfrak{p} \cdot \mathfrak{q}$.

Generating ideals with a short basis

How to generate $\mathfrak{p}, \mathfrak{q}$ with short basis of $\mathfrak{p} \cdot \mathfrak{q}$?

- Smart-Vercauteren: choose a random short $u \in R$, repeat until u generates a prime ideal.
- Ideal $\mathfrak{p} = u \cdot R$ is generated (over $\mathbb{Z}$) by

$$\mathbf{B} := \{u,\ u \cdot x,\ u \cdot x^2, \ldots,\ u \cdot x^{n-1}\}.$$

- Each element of $\mathbf{B}$ has norm at most $\|u\| \cdot \gamma_F$
  $\Rightarrow$ $\mathbf{B}$ is a short basis of $\mathfrak{p}$.
- Repeat to get $\mathfrak{q}$ with short generator w.
- $\{u \cdot w \cdot x^i\}_{i=0}^{n-1}$ is a short basis of $\mathfrak{p} \cdot \mathfrak{q}$.
- $\mathfrak{p}, \mathfrak{q}$ are principal prime ideals — not uniform.
  - Most ideals are not principal.

Signature length

How short are derived signatures?

Signature length

How short are derived signatures?

Define admissible function set $\mathcal{F}$ to be polynomials in $\mathbb{F}_p[x_1, \ldots, x_k]$ of degree $\leq d$ with coefficients in $[-y, y]$.

Signature length

How short are derived signatures?

Define admissible function set $\mathcal{F}$ to be polynomials in $\mathbb{F}_p[x_1, \ldots, x_k]$ of degree $\leq d$ with coefficients in $[-y, y]$.

Operation	Length expansion
Evaluate degree-d monomial	$\ell \mapsto \ell^d \cdot \gamma_F^{d-1}$
Multiply by coefficient in $[-y, y]$	$\ell \mapsto \ell \cdot y$
Sum of k vectors of length ℓ	$\ell \mapsto \ell \cdot k$

Signature length

How short are derived signatures?

Define admissible function set $\mathcal{F}$ to be polynomials in $\mathbb{F}_p[x_1, \ldots, x_k]$ of degree $\leq d$ with coefficients in $[-y, y]$.

Operation	Length expansion
Evaluate degree-d monomial	$\ell \mapsto \ell^d \cdot \gamma_F^{d-1}$
Multiply by coefficient in $[-y, y]$	$\ell \mapsto \ell \cdot y$
Sum of k vectors of length ℓ	$\ell \mapsto \ell \cdot k$

- Signatures on original messages m_i have length $< \beta\sqrt{n}$
  $\Rightarrow$ signature on $f(m_1, \ldots, m_k)$ has length $< (\beta\sqrt{n})^d \cdot \gamma_f^{d-1} \cdot y \cdot k$.

Signature length

How short are derived signatures?

Define admissible function set $\mathcal{F}$ to be polynomials in $\mathbb{F}_p[x_1, \ldots, x_k]$ of degree $\leq d$ with coefficients in $[-y, y]$.

Operation	Length expansion
Evaluate degree-d monomial	$\ell \mapsto \ell^d \cdot \gamma_F^{d-1}$
Multiply by coefficient in $[-y, y]$	$\ell \mapsto \ell \cdot y$
Sum of k vectors of length ℓ	$\ell \mapsto \ell \cdot k$

- Signatures on original messages m_i have length $< \beta\sqrt{n}$
  $\Rightarrow$ signature on $f(m_1, \ldots, m_k)$ has length $< (\beta\sqrt{n})^d \cdot \gamma_f^{d-1} \cdot y \cdot k$.
- If $\beta, \gamma_F, k, y \in \text{poly}(n)$ and $d = O(1)$, then derived signature length is $\text{poly}(n)$. (p is exponential in n)

Signature length

How short are derived signatures?

Define admissible function set $\mathcal{F}$ to be polynomials in $\mathbb{F}_p[x_1, \ldots, x_k]$ of degree $\leq d$ with coefficients in $[-y, y]$.

Operation	Length expansion
Evaluate degree-d monomial	$\ell \mapsto \ell^d \cdot \gamma_F^{d-1}$
Multiply by coefficient in $[-y, y]$	$\ell \mapsto \ell \cdot y$
Sum of k vectors of length ℓ	$\ell \mapsto \ell \cdot k$

- Signatures on original messages m_i have length $< \beta\sqrt{n}$ $\Rightarrow$ signature on $f(m_1, \ldots, m_k)$ has length $< (\beta\sqrt{n})^d \cdot \gamma_f^{d-1} \cdot y \cdot k$.
- If $\beta, \gamma_F, k, y \in \text{poly}(n)$ and $d = O(1)$, then derived signature length is poly(n). (p is exponential in n)
- For fixed n, bit length of derived signatures is linear in d, logarithmic in k.

Security of polynomial scheme

What is security based on?

Security of polynomial scheme

What is security based on?

Analysis of linearly homomorphic scheme also applies here:

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero element of $\mathfrak{q}$.

Security of polynomial scheme

What is security based on?

Analysis of linearly homomorphic scheme also applies here:

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero element of $\mathfrak{q}$.

- $\mathfrak{q}$ is a principal prime ideal.
  - Producing a short generator of arbitrary $\mathfrak{q}$ is a classical problem in algorithmic number theory.

Security of polynomial scheme

What is security based on?

Analysis of linearly homomorphic scheme also applies here:

Theorem

An adversary that wins the security game (in the random oracle model) can be used to compute a short nonzero element of $\mathfrak{q}$.

- $\mathfrak{q}$ is a principal prime ideal.
  - Producing a short generator of arbitrary $\mathfrak{q}$ is a classical problem in algorithmic number theory.
- Distribution of $\mathfrak{q}$ not well-studied.
  - Want $\mathfrak{q}$ in distribution that admits a worst-case reduction.

Open questions

1. Show that derived signatures are private.
   (i.e., leak no information about original messages)
   - Linearly homomorphic signatures are private.
   - Proof technique fails for polynomials.

Open questions

1. Show that derived signatures are private.
   (i.e., leak no information about original messages)
   - Linearly homomorphic signatures are private.
   - Proof technique fails for polynomials.

2. Remove random oracle from security proof.
   - Work in progress.

Open questions

1. Show that derived signatures are private.
   (i.e., leak no information about original messages)
   - Linearly homomorphic signatures are private.
   - Proof technique fails for polynomials.
2. Remove random oracle from security proof.
   - Work in progress.
3. Reduce security to worst-case problems on ideal lattices.
   - Generate ideals from uniform distribution (not all principal).
   - Adapt Gentry's random self-reduction.

Open questions

1. Show that derived signatures are private.
   (i.e., leak no information about original messages)
   - Linearly homomorphic signatures are private.
   - Proof technique fails for polynomials.
2. Remove random oracle from security proof.
   - Work in progress.
3. Reduce security to worst-case problems on ideal lattices.
   - Generate ideals from uniform distribution (not all principal).
   - Adapt Gentry's random self-reduction.
4. Fully homomorphic signatures!
   - Adapt "bootstrapping" approach???

Open questions

1. Show that derived signatures are private.
   (i.e., leak no information about original messages)
   - Linearly homomorphic signatures are private.
   - Proof technique fails for polynomials.
2. Remove random oracle from security proof.
   - Work in progress.
3. Reduce security to worst-case problems on ideal lattices.
   - Generate ideals from uniform distribution (not all principal).
   - Adapt Gentry's random self-reduction.
4. Fully homomorphic signatures!
   - Adapt "bootstrapping" approach???

Thank you!

Thanks also to Chris Peikert for help with graphics.